



The Policy RAG Agent

Explained in Plain English — No Technical Background Needed

How our Prior Authorization AI decides whether a medical treatment meets the insurance company's rules — what it does, how it does it, and which technology is behind it.

Prior Authorization AI · July 2026 · Document for non-technical readers

What problem does this system solve?

Before a hospital can perform many treatments — a knee replacement, a heart procedure, an expensive medication — the insurance company must approve it first. This approval is called a **Prior Authorization**.

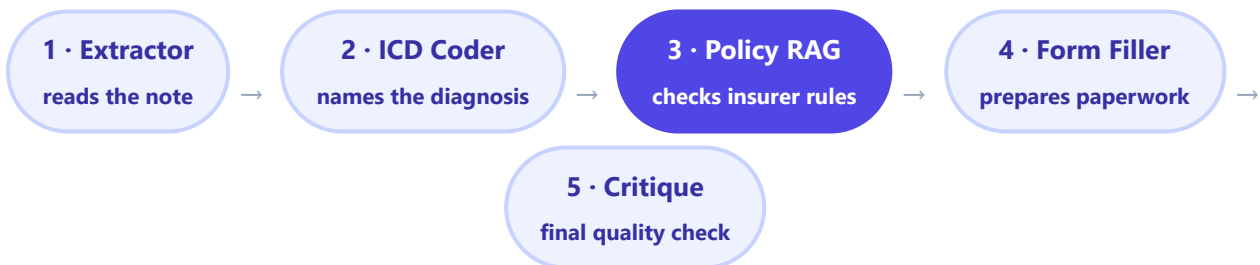
Today this is mostly manual work: someone reads the doctor's note, looks up the right insurance policy, checks the note against every rule in that policy, and fills in a long form. It is slow (days, sometimes weeks), repetitive, and easy to get wrong.

What our system does

It reads the doctor's note and produces an insurer-ready decision in well under a second — and whenever it is not sure, it stops and hands the case to a human instead of guessing.

The assembly line: five AI workers

The system works like an assembly line with five specialised workers ("agents"). Each one does a single job and passes its result to the next:



This document is about worker number **3 — the Policy RAG Agent**, the one that decides whether the treatment meets the insurance company's rules.

Meet the Policy RAG Agent — a librarian with a checklist

The easiest way to understand this agent is to picture **two people in one**:

The Librarian

The insurance company has a library of **32 policy documents** — one for knee replacements, one for heart procedures, one for biologic drugs, and so on. Given a patient's case, the librarian's job is to walk into that library and pull out the **one policy document that applies to this exact treatment**.

The Inspector

Once the right policy is found, the inspector opens it. Inside is a checklist of rules — for example: *"the condition must be severe"*, *"the patient must have tried simpler treatments for at least 12 weeks"*, *"an X-ray or scan must confirm the problem"*. The inspector ticks each rule the patient's note satisfies, and writes down every rule it does not.

"**RAG**" stands for **Retrieval-Augmented Generation** — a fancy phrase for a simple idea: *instead of letting an AI answer from memory, first go and **retrieve** the actual reference documents, and base the answer on them*. Exactly what a careful human would do: never approve from memory — always open the policy binder first.

PART 3

How it works, step by step

- 1 It receives the patient's facts — with privacy already protected.** By the time the case reaches this agent, the patient's name, date of birth, address, phone number and record number have already been removed by the first agent. The Policy RAG agent only ever sees medical facts: the diagnosis, the requested treatment, the symptoms, what has already been tried, and any imaging results.
- 2 It searches the policy library.** It compares the words in the patient's case against the keywords of all 32 policies — like scanning the labels on a filing cabinet. Each policy gets a **match score**: how many important words overlap, plus how closely phrases resemble each other (so "total knee replacement" still finds the knee policy even if the note words it slightly differently). The policy with the highest score wins.

3

It checks every rule in the winning policy. Each policy contains a checklist. There are three kinds of rules, and each is checked mechanically — the same way every single time:

RULE TYPE	PLAIN-ENGLISH MEANING	HOW IT IS CHECKED
Severity words	"The condition must be serious enough"	Looks for words like <i>severe</i> , <i>end-stage</i> , <i>advanced</i> in the case facts
Simpler treatment first	"Cheaper options must have been tried long enough"	Compares the documented weeks of therapy against the policy's minimum (e.g. 12 weeks)
Imaging proof	"A scan or X-ray must confirm the diagnosis"	Checks whether the note documents an imaging result

4

It writes an honest report card. The output lists **which rules passed**, **which rules are missing**, a **confidence score**, and a one-line explanation of how it reached its conclusion. If even one rule is unmet, the case is **not rejected** — it is routed to a human reviewer with a precise note about exactly what is missing.

A real example

Patient case: "75-year-old, end-stage degenerative joint disease, no conservative therapy documented, imaging confirms, requesting total knee replacement"

Library search → best match: POL-ORTHO-TKR-001 "Total Knee Replacement Policy"

Checklist:

✓ Severity documented "end-stage" found
 ✓ Imaging confirms X-ray result present
 ✗ 12+ weeks conservative therapy . nothing documented

Result: 2 of 3 rules met → case sent to a HUMAN REVIEWER

Reason given: "Missing: conservative therapy failed for at least 12 weeks"

Notice what did *not* happen: the system did not guess, did not approve anyway, and did not reject the patient. It explained exactly what was missing and asked a person to decide.

The jargon, translated — and what this system actually uses

When people talk about RAG systems you will hear words like *chunking*, *embeddings*, *vector databases* and *semantic search*. Here is what each one means — and, honestly, whether this project uses it. (Spoiler: our policy library is small — 32 documents — so we deliberately chose a simpler method that is easier to explain and audit. The fancy tools become worthwhile when the library grows to thousands of documents.)

TERM	PLAIN-ENGLISH MEANING	USED HERE?
Retrieval (the "R" in RAG)	Go and fetch the actual reference document before answering — never answer from memory.	✓ Yes. Every case fetches a real policy from the library first.
Keyword & fuzzy matching	Compare the words in the case against the words that describe each policy; "fuzzy" means near-misses still count ("knee arthroplasty" ≈ "knee replacement").	✓ Yes — this is our search engine. Simple, fast, and every match can be explained word by word.
Chunking	Cutting long documents into bite-sized pieces so a search can point at the exact paragraph, not just the whole file.	Not needed. Each policy is already stored as a small structured card (title, keywords, checklist), so there is nothing long to cut up.
Embeddings	Turning a sentence into a list of numbers that captures its <i>meaning</i> , so a computer can measure how similar two sentences are even when they share no words.	Not currently. With 32 well-labelled policies, keyword matching finds the right one reliably — and its reasoning is fully auditable. Embeddings are the natural upgrade if the library grows large.
Vector database	A special filing cabinet built to store those number-lists and find the closest ones very quickly.	Not currently — no embeddings, so nothing to store. The policies live in a simple structured file.
Semantic search	Searching by meaning rather than exact words ("chest pain drug" finds "angina medication").	Partly. Fuzzy matching gives a light version of this. Full semantic search would come with embeddings.
LLM (Large Language Model)	The kind of AI that reads and writes text, like GPT-4o.	Optional, and never for decisions. Azure OpenAI GPT-4o can be switched on to help read messy doctor's notes (step 1 only). All policy checking is done by fixed rules, so the same case always gets the same answer.

Why the simple way is a feature, not a shortcut

In healthcare, an approval decision must be **explainable and repeatable**. Our rule-based checks mean: the same note always produces the same result, every tick on the checklist can be traced to actual words in the note, and an auditor can verify any decision by hand. A purely AI-driven judgement could not promise any of that.

PART 5

The technology behind it (one-line each)

TECHNOLOGY	WHAT IT IS	WHAT IT DOES IN THIS PROJECT
Python	A popular programming language	Everything is written in it
Pydantic	A "form validator" for data	Guarantees every agent hands over complete, correctly-shaped information — a typo or missing field is caught instantly
FastAPI + Uvicorn	A web server toolkit	Powers the dashboard you open in the browser
NetworkX knowledge graph	A map of connected medical facts	Stores the ICD-10 diagnosis codes the ICD Coder matches against
difflib fuzzy matching	A built-in text-similarity tool	The "close enough" word matching used by the policy search
JSON policy library	A structured text file	Holds all 32 insurer policies with their keywords and checklists
Azure OpenAI (GPT-4o)	Microsoft-hosted AI language model	<i>Optional:</i> helps read free-text doctor's notes. Never checks policies, never approves anything
Audit trace logger	A flight recorder	Writes down every step, timestamp, confidence and reason for every case

How the confidence score is calculated

The agent's confidence blends two questions, weighted so that the checklist matters most:

- 30% — "How sure am I that I pulled the right policy off the shelf?" (library match score)
- 70% — "How many of that policy's rules did the patient actually meet?" (checklist score)

A case matching the perfect policy but meeting only 1 of 3 rules gets a low score — and low scores are exactly what route a case to a human.

PART 6

Safety promises, in plain words

PROMISE	HOW IT IS KEPT
Patient privacy	Names, birth dates, addresses, phone numbers and record numbers are masked before <i>any</i> AI reasoning happens — and every later agent re-checks its own input for leaks
No guessing	Unclear diagnosis? Conflicting left/right side? Missing policy requirement? The case stops and goes to a human — with a written reason
Every decision explainable	Rule-based checks + an audit log means any decision can be reconstructed and verified afterwards
Independent double-check	A fifth agent (the Critique Agent) re-verifies every completed package — field by field — before it leaves the system

Prior Authorization AI · Policy RAG Agent explainer · Generated July 2026. Companion file: **policy-rag-flow.pptx** — a slide deck with the visual flow diagram of this agent.